

Multi-Initial-State Simulation in SliQSim

Daniel
Final Project Report
Taipei, Taiwan

ABSTRACT

This project extends SliQSim with simultaneous simulation of multiple labeled initial states. The method introduces Boolean selector variables that encode the initial-state label. The quantum circuit is then applied once to the combined Boolean decision diagram (BDD) representation. After the circuit is evaluated, each labeled output state is recovered by cofactoring the final BDDs with the corresponding selector assignment. The implementation supports computational-basis initial states, dense arbitrary-amplitude initial states, sparse arbitrary-amplitude initial states, repeated access to selected output labels, sequential verification, probability-tolerant comparison, BDD dumping, and a comparison script against MQT DDSIM. Experiments show that the all-in-one method produces the same support and matching probabilities as sequential SliQSim simulation on the tested circuits. Against DDSIM, the small statevector tests agree within the fixed-point precision used by SliQSim, and finite-shot sampling tests agree within expected sampling variation. For a 100-qubit sparse arbitrary-amplitude input, DDSIM cannot construct the required dense initial state vector, while the SliQSim sparse BDD input remains executable.

1 OBJECTIVE

The objective is to add multi-initial-state simulation to SliQSim. Given a quantum circuit U and a set of labeled initial states

$$\mathcal{I} = \{(\lambda_0, |\psi_0\rangle), (\lambda_1, |\psi_1\rangle), \dots, (\lambda_{m-1}, |\psi_{m-1}\rangle)\},$$

the simulator should evaluate all states through the same circuit without rerunning the circuit m times. After this shared simulation, the user should be able to select a label λ_i and use the existing SliQSim output functions, including sampling, statevector extraction, and probability or statistical queries.

2 ORIGINAL SLIQSIM STATE REPRESENTATION

Let the quantum register have n qubits. A computational basis state is indexed by a Boolean vector

$$q = (q_0, q_1, \dots, q_{n-1}) \in \{0, 1\}^n.$$

SliQSim represents a quantum state as a set of Boolean functions rather than as an explicit length- 2^n array. For each basis assignment q , the amplitude is represented by SliQSim's fixed-point integer arithmetic. Internally, SliQSim stores four amplitude components and r integer bits per component:

$$B_{c,b}(q) \in \{0, 1\}, \quad c \in \{0, 1, 2, 3\}, \quad b \in \{0, 1, \dots, r-1\}.$$

The implementation stores these roots in

$$\text{All_Bdd}[c][b].$$

Each root is a BDD over the qubit variables. The integer value of one component is reconstructed from its bit slices. SliQSim also maintains a global scaling parameter k , which records powers of

$\sqrt{2}$ introduced by gates such as Hadamard and rotations. Therefore the simulator does not store one floating-point complex number per basis state. It stores bit-sliced Boolean functions whose evaluations reconstruct the amplitude under SliQSim's original arithmetic model.

This project does not replace the original gate arithmetic. All gate functions still update the same $\text{All_Bdd}[c][b]$ roots by BDD operations such as cofactor, conjunction, disjunction, and exclusive-or.

3 MULTI-INITIAL-STATE ENCODING

3.1 Selector Variables

For m initial states, the implementation introduces

$$\ell = \lceil \log_2 m \rceil$$

selector variables

$$s = (s_0, s_1, \dots, s_{\ell-1}) \in \{0, 1\}^\ell.$$

These variables are allocated after the qubit variables in the CUDD manager. If the circuit has n qubits, then qubit variables use indices $0, \dots, n-1$, and selector variables use indices

$$n, \dots, n + \ell - 1.$$

The CUDD manager is initialized with

$$n + \ell$$

primary BDD variables. For label index i , let $\text{bin}_\ell(i)$ be the ℓ -bit binary representation of i . Define the selector cube

$$C_i(s) = \bigwedge_{j=0}^{\ell-1} \begin{cases} s_j, & \text{bin}_\ell(i)_j = 1, \\ \neg s_j, & \text{bin}_\ell(i)_j = 0. \end{cases}$$

This cube is true only for the selector assignment corresponding to label λ_i .

3.2 Combined Initial State

Let $\alpha_i(q)$ be the amplitude of basis state q in initial state $|\psi_i\rangle$. The combined multi-initial-state amplitude function is

$$\Psi(q, s) = \begin{cases} \alpha_i(q), & s = \text{bin}_\ell(i), \\ 0, & s \notin \{\text{bin}_\ell(0), \dots, \text{bin}_\ell(m-1)\}. \end{cases}$$

Equivalently,

$$\Psi(q, s) = \sum_{i=0}^{m-1} C_i(s) \alpha_i(q).$$

For each fixed-point component c and bit b , the BDD root is constructed as the union of all labeled terms whose encoded bit is one:

$$B_{c,b}^{\text{multi}}(q, s) = \bigvee_{i=0}^{m-1} \left(C_i(s) \wedge B_{c,b}^{(i)}(q) \right).$$

This is the central construction. It stores all labels in one set of BDD roots, but the labels remain disjoint because the selector cubes C_i are mutually exclusive.

3.3 Basis Initial States

For a basis state string $x_i \in \{0, 1\}^n$, the amplitude is one on that assignment and zero elsewhere:

$$\alpha_i(q) = \begin{cases} 1, & q = x_i, \\ 0, & q \neq x_i. \end{cases}$$

The BDD term inserted into the real component is

$$C_i(s) \wedge \bigwedge_{j=0}^{n-1} \begin{cases} q_j, & (x_i)_j = 1, \\ \neg q_j, & (x_i)_j = 0. \end{cases}$$

The value one is encoded using SliQSim's fixed-point integer representation.

3.4 Dense Arbitrary-Amplitude Initial States

A dense arbitrary-amplitude line gives all 2^n amplitudes in computational-basis order:

$$\lambda_i \quad a_0 \quad a_1 \quad \dots \quad a_{2^n-1}.$$

Each value $a_t \in \mathbb{C}$ is parsed as

$$a_t = \text{Re}(a_t) + i \text{Im}(a_t).$$

The implementation encodes real and imaginary parts by fixed-point scaling. Let the number of fractional bits be

$$f = \min(24, r - 2).$$

Then a real number x is encoded as

$$\hat{x} = \text{round}(x2^f).$$

The simulator sets

$$k = 2f,$$

so that the stored integer is interpreted with scale 2^{-f} . Therefore the decoded value is approximately

$$x \approx \hat{x}2^{-f}.$$

This approximation is the source of small probability differences against floating-point DDSIM statevectors.

3.5 Sparse Arbitrary-Amplitude Initial States

Dense input is impossible for large n . For example, $n = 100$ would require 2^{100} amplitudes. Therefore the implementation also supports sparse entries:

$$\lambda_i \quad x_{i,0} : a_{i,0} \quad x_{i,1} : a_{i,1} \quad \dots$$

where each $x_{i,t} \in \{0, 1\}^n$ is a basis string and $a_{i,t} \in \mathbb{C}$ is its amplitude. The BDD contribution for each sparse term is

$$C_i(s) \wedge Q_{i,t}(q),$$

where $Q_{i,t}$ is the basis cube for $x_{i,t}$. Only nonzero terms are inserted. This is the input format used for the 100-qubit Bernstein–Vazirani experiment.

4 WHY GATE IMPLEMENTATIONS DO NOT CHANGE

The selector variables are not quantum variables. They are label variables. Every quantum gate in SliQSim acts on qubit variables only. A gate G on the quantum register is applied to the combined state as

$$I_{\text{selector}} \otimes G.$$

Thus, if the initial combined state is

$$\Psi(q, s) = \sum_i C_i(s) \alpha_i(q),$$

then after applying circuit U the result is

$$\Psi'(q, s) = \sum_i C_i(s) (U \alpha_i)(q).$$

The selector cube $C_i(s)$ is unchanged.

At the BDD level, the original gate code performs operations such as

$$\text{Cofactor}(B_{c,b}, q_t), \quad \text{Cofactor}(B_{c,b}, \neg q_t), \quad B_{c,b} \wedge q_t, \quad B_{c,b} \vee q_t, \quad B_{c,b} \oplus q_t.$$

The variable q_t is always a qubit variable with index less than n . The selector variables have indices at least n . Therefore the gate code transforms the dependence on q while preserving the dependence on s . This is why no gate-specific rewrite is needed.

5 RECOVERING ONE OUTPUT STATE

After the circuit has been simulated once, the user selects a label λ_i . The implementation constructs the selector cube $C_i(s)$ and cofactors every BDD root:

$$B_{c,b}^{(i),\text{out}}(q) = \text{Cofactor}\left(B_{c,b}^{\text{multi,out}}(q, s), C_i(s)\right).$$

Because the selector cubes are disjoint, this operation extracts exactly the output state for label λ_i :

$$\Psi'_i(q) = (U \alpha_i)(q).$$

After this projection, SliQSim's existing functions can be called without modification. The implementation temporarily replaces `All_Bdd` with the selected cofactored roots, performs sampling, statevector extraction, or query evaluation, and then restores the original multi-state roots. This permits repeated access to different labels without rerunning the quantum circuit.

6 MERGED BDD FOR MEASUREMENT

The original SliQSim measurement procedure merges the bit-sliced roots into one large BDD called `bigBDD`. With r integer bits and $w = 4$ amplitude components, the merged BDD needs auxiliary variables to select one component and one integer bit. The number of auxiliary variables is

$$\lceil \log_2 w \rceil + \lceil \log_2 r \rceil.$$

With the default $w = 4$ and $r = 32$, this is

$$\lceil \log_2 4 \rceil + \lceil \log_2 32 \rceil = 2 + 5 = 7.$$

These auxiliary variables are not qubits and not initial-state selectors. They only encode which amplitude component and which fixed-point bit are being traversed during probability computation.

7 VERIFICATION METHOD

7.1 Sequential SliQSim Verification

The implementation includes a sequential verification mode. For each selected label λ_i , it constructs a simulator containing only that one initial state, runs the same circuit, and records the output. This produces the reference set

$$\{U|\psi_i\rangle\}_{i \in S},$$

where S is the selected label set. The all-in-one output is compared against this sequential output.

For statevectors, the comparison is performed by converting amplitudes to probabilities. For sampling results, counts are first normalized:

$$\hat{p}_{i,y} = \frac{N_{i,y}}{N_i},$$

where $N_{i,y}$ is the number of times label i produced output bitstring y , and N_i is the shot count for label i .

7.2 Support Check

Before comparing numeric probabilities, the verifier checks support equality. For each label i , define

$$\text{supp}(p_i) = \{y : p_{i,y} > 0\}.$$

The support check requires

$$\text{supp}(p_i^{\text{multi}}) = \text{supp}(p_i^{\text{seq}})$$

for all selected labels. If an output bitstring appears in one mode but not the other, the report identifies the missing label and missing output.

7.3 Probability Difference

After the support check, the maximum absolute probability difference is computed:

$$\Delta_{\max} = \max_{i,y} |p_{i,y}^{\text{multi}} - p_{i,y}^{\text{reference}}|.$$

For finite-shot sampling, nonzero differences are expected because the two simulators draw random samples independently. Therefore the verification command accepts a tolerance. In the report experiments, statevector comparisons show differences below 10^{-6} , while sampling comparisons show differences on the order expected from 10,000 shots.

8 IMPLEMENTED USER INTERFACE

The implementation adds the following user-facing options:

- `-init_states` FILE: read labeled initial states.
- `-select_init` LABELS: access all labels or a comma-separated subset.
- `-sequential_init`: run selected labels one by one.
- `-verify_init`: run all-in-one and sequential modes and compare them.
- `-verify_tol` VALUE: probability tolerance for verification.
- `-dump_bdds` PREFIX: export Graphviz DOT files for initial BDD slices and final merged BDDs.

The comparison script is:

```
tools/compare_simulators.py.
```

It runs SliQSim all-in-one, SliQSim sequential, and MQT DDSIM sequential simulation. DDSIM is run label by label because it does not implement the selector-variable encoding. Each DDSIM label is executed in a child process with a hard wall-time limit `-ddsim-timeout`. If DDSIM cannot construct a dense initial state, or if the label exceeds the timeout, the label is reported as aborted.

9 EXPERIMENTAL SETUP

All experiments were run on the same local workstation using the current SliQSim executable in this repository. The random seed was fixed to zero. Unless otherwise stated, SliQSim used the default integer bit size $r = 32$. Sampling experiments used 10,000 shots. DDSIM was called through the Python package `mqt.ddsim`. The reported peak memory values are resident-set-size measurements. SliQSim also reports its own peak memory and maximum BDD node count.

9.1 Test Cases

The report uses the following circuits and initial-state files:

- `examples/bell_state.qasm` with `init_state/init_states.txt`: two basis labels.
- `examples/bell_state_measure.qasm` with `init_state/init_states.txt`: two basis labels and measurement.
- `examples/report_entangle3.qasm` with `examples/report_basis4_init.txt`: four basis labels on a 3-qubit circuit.
- `examples/report_entangle3.qasm` with `examples/report_sparse_complex_init.txt`: four sparse complex arbitrary labels.
- `examples/report_entangle3_measure.qasm` with `examples/report_sparse_complex_init.txt`: measured sparse complex arbitrary labels.
- `examples/bv100.qasm` with `examples/bv100_sparse_arbitrary_init.txt`: three selected sparse arbitrary labels from a 100-qubit benchmark.

10 EXPERIMENTAL RESULTS

10.1 Runtime and Size Statistics

10.2 Correctness and Probability Agreement

The statevector comparisons have differences below 10^{-6} . These differences are consistent with SliQSim's fixed-point encoding of decimal amplitudes. The sampling comparisons have larger differences because the compared simulators perform finite-shot random sampling. With 10,000 shots, differences on the order of 10^{-2} are plausible for probabilities near $1/2$ or $1/4$.

For BV100, DDSIM did not produce reference probabilities because the arbitrary sparse 100-qubit states would need to be expanded into dense vectors of length 2^{100} . The comparison script therefore reports the DDSIM labels as aborted. The SliQSim multi-vs-sequential comparison still has matching labels and matching

Table 1: Runtime, memory, and BDD size statistics. “Multi” is SliQSim all-in-one multi-initial-state simulation. “Seq.” is SliQSim sequential simulation. “DDSIM” is MQT DDSIM sequential simulation.

Case	Mode	Wall s	RSS bytes	SliQSim s	SliQSim mem	Gates	Max nodes
Bell statevector	Multi	0.019646	3,305,472	0.008155	18,219,008	2	6
Bell statevector	Seq.	0.021139	7,757,824	0.013330	19,660,800	–	–
Bell statevector	DDSIM	0.182977	114,528,256	–	–	–	–
Bell sampling	Multi	0.030328	11,702,272	0.025338	18,087,936	2	21
Bell sampling	Seq.	0.041724	13,799,424	0.031382	19,529,728	–	–
Bell sampling	DDSIM	0.172827	114,704,384	–	–	–	–
3q basis statevector	Multi	0.020347	11,567,104	0.009299	18,309,120	6	23
3q basis statevector	Seq.	0.021183	6,721,536	0.016246	19,488,768	–	–
3q basis statevector	DDSIM	0.358148	114,315,264	–	–	–	–
3q complex statevector	Multi	0.019701	3,330,048	0.007975	18,337,792	6	65
3q complex statevector	Seq.	0.021281	5,468,160	0.016041	19,779,584	–	–
3q complex statevector	DDSIM	0.446059	114,520,064	–	–	–	–
3q complex sampling	Multi	0.081051	13,500,416	0.068653	18,350,080	6	138
3q complex sampling	Seq.	0.082979	13,582,336	0.075722	19,791,872	–	–
3q complex sampling	DDSIM	0.782672	114,597,888	–	–	–	–
BV100 sparse sampling	Multi	4.091575	17,227,776	4.075810	18,317,312	299	4,272
BV100 sparse sampling	Seq.	0.626792	37,425,152	0.622212	37,425,152	–	–
BV100 sparse sampling	DDSIM	0.083774	114,606,080	–	–	–	–

Table 2: Probability comparison results. Support match means that both compared modes produced the same set of output bitstrings for every selected label.

Case	Comparison	Same labels	Support match	$\Delta_{\max}$	Max label
Bell statevector	Multi vs Seq.	true	true	0	–
Bell statevector	Multi vs DDSIM	true	true	3.09×10^{-7}	one
Bell sampling	Multi vs Seq.	true	true	0.0006	one
Bell sampling	Multi vs DDSIM	true	true	0.0028	zero
3q basis statevector	Multi vs Seq.	true	true	0	–
3q basis statevector	Multi vs DDSIM	true	true	5.52×10^{-7}	high
3q complex statevector	Multi vs Seq.	true	true	0	–
3q complex statevector	Multi vs DDSIM	true	true	3.68×10^{-7}	four_term
3q complex sampling	Multi vs Seq.	true	true	0.0125	four_term
3q complex sampling	Multi vs DDSIM	true	true	0.0161	phi_plus
BV100 sparse sampling	Multi vs Seq.	true	true	0.0450	arb01
BV100 sparse sampling	Multi vs DDSIM	false	false	0.2790	arb02

support. Its maximum sampled probability difference is 0.045 for 1,000 shots, which is a finite-shot sampling difference rather than a statevector disagreement.

10.3 DDSIM Per-Label Statistics

11 INTERPRETATION

The experiments support four conclusions.

First, the selector-variable construction is functionally correct for the tested circuits. SliQSim all-in-one and SliQSim sequential

simulation always had the same selected label set and the same output support. For statevector experiments, their probability distributions were identical after parsing. For sampling experiments, the differences were finite-shot sampling differences.

Second, the gate implementations do not need to be modified. Since the gate code only cofactors and combines on qubit variables, selector variables are preserved automatically. This realizes the operator $I_{\text{selector}} \otimes U$.

Third, sparse arbitrary-amplitude input is necessary for large circuits. DDSIM can initialize small arbitrary states by expanding

Table 3: DDSIM per-label summary for representative non-aborted cases. Backend time is the time reported by DDSIM. Total wall time also includes Python, Qiskit circuit construction, and process overhead.

Case	Label set	Total wall s per label	Backend s per label	Depth	Operations
Bell statevector	zero, one	0.085–0.086	0.041–0.047	2–3	2–3
Bell sampling	zero, one	0.085–0.086	0.042	3–4	4–5
3q basis statevector	four labels	0.075–0.096	0.043–0.053	5–6	6–9
3q complex statevector	four labels	0.105–0.116	0.045–0.048	15	20
3q complex sampling	four labels	0.188–0.199	0.130–0.143	16	23

the vector and prepending an initialization circuit. For 100 qubits, dense expansion is infeasible. SliQSim can still construct the sparse BDD terms because it only inserts the nonzero basis cubes.

Fourth, all-in-one multi-initial-state simulation is not guaranteed to be faster than sequential simulation for every input. The all-in-one BDD includes selector variables and may increase BDD size. It is most useful when the selected initial states share BDD structure during the circuit. When there is little sharing or when only a few labels are selected, sequential simulation can be faster. The benefit is therefore structural, not automatic.

12 LIMITATIONS

The implementation has the following limitations:

- Decimal arbitrary amplitudes are fixed-point approximations controlled by `-r`. They are not exact real numbers.
- Dense arbitrary-amplitude input is only practical for small n because it requires 2^n entries.
- BDD traversal is polynomial in the number of BDD nodes, but the BDD size itself can be exponential in the number of qubits.
- DDSIM is not an all-in-one multi-initial-state baseline. It is used as a sequential external reference.
- Finite-shot sampling comparisons are random experiments. Probability differences decrease statistically as the shot count increases, but exact equality is not expected.

13 REPRODUCIBILITY COMMANDS

The main commands used to generate the tables are shown below.

```
python3 tools/compare_simulators.py \
--qasm examples/bell_state.qasm \
--init-states init_state/init_states.txt \
--type 1 --seed 0 --ddsim-timeout 10 \
--json-out /tmp/report_bell_basis.json

python3 tools/compare_simulators.py \
--qasm examples/bell_state_measure.qasm \
--init-states init_state/init_states.txt \
--type 0 --shots 10000 --seed 0 --ddsim-timeout 10 \
--json-out /tmp/report_bell_measure.json

python3 tools/compare_simulators.py \
--qasm examples/report_entangle3.qasm \
--init-states examples/report_basis4_init.txt \
--type 1 --seed 0 --ddsim-timeout 10 \
--json-out /tmp/report_entangle3_basis.json
```

```
python3 tools/compare_simulators.py \
--qasm examples/report_entangle3.qasm \
--init-states examples/report_sparse_complex_init.txt \
--type 1 --seed 0 --ddsim-timeout 10 \
--json-out /tmp/report_entangle3_complex.json

python3 tools/compare_simulators.py \
--qasm examples/report_entangle3_measure.qasm \
--init-states examples/report_sparse_complex_init.txt \
--type 0 --shots 10000 --seed 0 --ddsim-timeout 10 \
--json-out /tmp/report_entangle3_complex_measure.json

python3 tools/compare_simulators.py \
--qasm examples/bv100.qasm \
--init-states examples/bv100_sparse_arbitrary_init.txt \
--select-init arb00,arb01,arb02 \
--type 0 --shots 1000 --seed 0 --ddsim-timeout 1 \
--json-out /tmp/report_bv100_sparse3.json
```

14 CONCLUSION

This project implements multi-initial-state simulation in SliQSim by adding selector variables to the BDD representation of the initial state. The mathematical construction is a direct sum over mutually exclusive selector cubes. The original gate operations remain valid because each gate acts only on quantum variables, so the selector variables are preserved as labels. Output states are recovered by co-factoring with the selected label cube. The implementation also adds arbitrary-amplitude input, sparse input, label selection, sequential verification, probability-tolerant comparison, BDD dumping, and DDSIM comparison tooling. The experimental results show agreement between all-in-one and sequential SliQSim simulation, close agreement with DDSIM on small cases, and a clear scalability advantage of sparse BDD initialization over DDSIM dense initialization for the 100-qubit arbitrary-state benchmark.